

Decision *Records*

12 ključnih arhitekturnih odločitev

Zbirka **12 ADR-jev** z kontekstom, alternativami in posledicami. Vsak ADR odgovarja na "zakaj smo se odločili za X in ne Y". Pokriva: Next.js, Neon, Prisma, multi-tenant, PIN auth, FURS, Service Worker, design system, Stripe, audit log, Vercel in i18n.

12

ADR-JEV

60+

ALTERNATIV

5

LET (2025)

100%

ACCEPTED

Kazalo vsebine

1. Uvod v ADR	4
1.1 Struktura vsakega ADR-ja	4
1.2 Seznam vseh ADR-jev	4
2. ADR-001: Next.js 16 + React 19	5
Kontekst	5
Odločitev	5
Alternative	5
Posledice	5
3. ADR-002: Neon PostgreSQL	7
Kontekst	7
Odločitev	7
Alternative	7
Posledice	7
4. ADR-003: Prisma ORM	9
Kontekst	9
Odločitev	9
Alternative	9
Posledice	9
5. ADR-004: Multi-tenant arhitektura	10
Kontekst	10
Odločitev	10
Alternative	10

Posledice	11
6. ADR-005: PIN-based auth	12
Kontekst	12
Odločitev	12
Alternative	12
Posledice	13
7. ADR-006: FURS OpenSSL CLI + Node crypto	14
Kontekst	14
Odločitev	14
Alternative	14
Posledice	15
8. ADR-007: Service Worker offline	16
Kontekst	16
Odločitev	16
Alternative	16
Posledice	17
9. ADR-008: Cascade palette V2	18
Kontekst	18
Odločitev	18
Alternative	18
Posledice	19
10. ADR-009: Stripe plačilni gateway	20
Kontekst	20

Odločitev	20
Alternative	20
Posledice	20
11. ADR-010: Chain hash audit log	22
Kontekst	22
Odločitev	22
Alternative	23
Posledice	23
12. ADR-011: Vercel hosting	24
Kontekst	24
Odločitev	24
Alternative	24
Posledice	24
13. ADR-012: next-intl za i18n	26
Kontekst	26
Odločitev	26
Alternative	26
Posledice	26
14. Skupne posledice in naslednji koraki	28
14.1 Skupne prednosti	28
14.2 Skupne slabosti	28
14.3 ADR-ji za ponovno pretehtavanje	28
14.4 Postopek za nove ADR-je	29

1. Uvod v ADR

Architecture Decision Records (ADR) so kratki dokumenti, ki beležijo ključne tehnične odločitve v projektu. Vsak ADR odgovarja na vprašanja: zakaj smo se odločili za X in ne Y? Kaj so bile alternative? Kakšne so posledice?

Ta zbirka vsebuje **12 ključnih ADR-jev** za RestaurantOS v1.0.0. ADR-ji so razvrščeni po datumu sprejema (od najstarejšega do najnovejšega). Vsak ADR ima standardno strukturo: kontekst, odločitev, alternative, posledice.

1.1 Struktura vsakega ADR-ja

- **ID:** ADR-XXX (zaporedna številka)
- **Naslov:** kratek opis odločitve
- **Status:** Accepted / Superseded / Deprecated
- **Datum:** kdaj je bila odločitev sprejeta
- **Kontekst:** zakaj smo odločali (problem, zahteve, omejitve)
- **Odločitev:** kaj smo se odločili (konkretna izbira)
- **Alternative:** kaj smo zavrnili in zakaj (vsaj 2-3 alternative)
- **Posledice:** good (prednosti), bad (slabosti), neutral (nevtralno)

1.2 Seznam vseh ADR-jev

ID	Naslov	Status	Datum
ADR-001	Next.js 16 + React 19 kot frontend framework	Accepted	2025-01-15
ADR-002	Neon PostgreSQL (serverless) kot glavna baza	Accepted	2025-01-20
ADR-003	Prisma ORM za databasno dostop	Accepted	2025-01-22
ADR-004	Multi-tenant arhitektura z locationId izolacijo	Accepted	2025-02-01
ADR-005	PIN-based auth namesto JWT/session	Accepted	2025-02-10
ADR-006	FURS modul z OpenSSL CLI + Node crypto fallback	Accepted	2025-02-15
ADR-007	Service Worker za offline mode (IndexedDB queue)	Accepted	2025-03-01
ADR-008	Cascade palette V2 za design system	Accepted	2025-03-15
ADR-009	Stripe kot primarni plačilni gateway	Accepted	2025-04-01
ADR-010	Chain hash audit log (SHA-256, nepopravljiv)	Accepted	2025-04-15
ADR-011	Vercel kot hosting platforma	Accepted	2025-05-01
ADR-012	next-intl za i18n (5 jezikov)	Accepted	2025-05-15

Tabela 1.1: Seznam vseh 12 ADR-jev

2. ADR-001: Next.js 16 + React 19

ADR-001	Next.js 16 + React 19 kot frontend framework	Accepted
Datum: 2025-01-15		

Kontekst

RestaurantOS potrebuje sodoben frontend framework, ki podpira: server-side rendering (SEO za landing page), API routes (backend), TypeScript, hitrost in dobro developer izkušnjo. Sistem mora delovati na tablicah (POS), mobilnih napravah (natakarji) in desktopih (admin).

Omejitve: ekipa ima izkušnje z React, projekt mora biti production-ready v 6 mesecih, hosting na Vercel (prednost za Next.js).

Odločitev

Izbrali smo Next.js 16 z React 19, ki ga hostamo na Vercel. Uporabljamo App Router (ne Pages Router), Server Components za performanco in API Routes za backend.

Alternative

Alternativa	Pros	Cons	Zakaj zavrnjena
Vue.js + Nuxt	Enostaven learning curve, dobra DX	Manjša skupnost, manj job kandidatov	Ekipa nima Vue izkušenj
SvelteKit	Najboljša performanca, majhen bundle	Majhna ekosistem, manj komponent	Premajhna skupnost za enterprise
Remix	Odlično za web standards	Manj integracij kot Next.js	Manj komponent v ekosistemu
Angular	Kompleten framework, enterprise-ready	Strma learning curve, težji	Prekompleksen za našo ekipo
Plain React (Vite)	Polna kontrola, enostaven	Brez SSR, API routes, file-based routing	Premalo funkcij v paketu

Tabela 2.1: Primerjava alternativ za ADR-001

Posledice

- **Good:** SSR za SEO, API routes (en paket), Vercel integracija, velika skupnost, enostaven hiring
- **Good:** Server Components zmanjšujejo bundle size, streaming SSR za hitrejši first paint

- **Good:** TypeScript first-class support, ecosystem (Radix UI, Tailwind, Prisma)
- **Bad:** Next.js 16 je nov (breaking changes morda), dokumentacija včasih zaostaja
- **Bad:** App Router je kompleksnejši od Pages Router (caching, revalidation)
- **Neutral:** Zaklenjeni smo v Vercel ekosistem (lahko selimo na self-hosted, a z naporom)

3. ADR-002: Neon PostgreSQL

ADR-002	Neon PostgreSQL (serverless) kot glavna baza	Accepted
Datum: 2025-01-20		

Kontekst

RestaurantOS potrebuje relacijsko bazo za transakcijske podatke (naročila, plačila, FURS računi). Zahtevane lastnosti: SQL (ne NoSQL), ACID transakcije, GDPR skladnost (EU hosting), avtomatski backupi, scale-to-zero za nizke stroške v začetku.

Omejitve: budget <100 EUR/mes za bazo v prvem letu, ekipa pozna PostgreSQL, prioriteta EU hosting za GDPR.

Odločitev

Izbrali smo Neon PostgreSQL (serverless PostgreSQL). Free plan za razvoj (0.5 GB), scale-up po potrebi. Connection pooling prek Neon pooler (port 5432). Branching za testne environmente.

Alternative

Alternativa	Pros	Cons	Zakaj zavrnjena
Supabase	Auth + DB + Storage v enem	Lock-in, manj fleksibilnosti	Preveč "magic", radi imamo kontrolo
PlanetScale (MySQL)	Odlično za branje, branching	MySQL (ne PostgreSQL), no foreign keys na free	No FK je deal-breaker
AWS RDS PostgreSQL	Polna kontrola, enterprise	Drago, ni serverless, complex setup	Predrago za začetek
Self-hosted PostgreSQL	Polna kontrola, brezplačno	Vzdrževanje, backup, security naša odgovornost	Ni dovolj časa za ops
MongoDB Atlas	Fleksibilna shema, dobra za dokumente	NoSQL (ne ACID), transakcije šibke	Potrebujemo ACID za plačila

Tabela 3.1: Primerjava alternativ za ADR-002

Posledice

- **Good:** Serverless (scale-to-zero), branching za test, free plan za začetek, EU hosting (GDPR)
- **Good:** Polno PostgreSQL (ACID, foreign keys, transakcije, full-text search, JSONB)

- **Good:** Avtomatski backupi (PITR 7 dni na free planu)
- **Bad:** Free plan omejen (0.5 GB storage, 100 compute hours/mes) - bo treba upgrade pri rasti
- **Bad:** Cold start (prva query po idle lahko traja 1-2s) - mitigiramo z keep-alive
- **Bad:** Vendor lock-in (Neon specifične funkcije)
- **Neutral:** Connection pooler je Poseganje (ne native PG), a deluje dobro

4. ADR-003: Prisma ORM

ADR-003	Prisma ORM za databasno dostop	Accepted
Datum: 2025-01-22		

Kontekst

Potreben je ORM za dostop do PostgreSQL iz Next.js. Zahteve: TypeScript-first, type safety, migracije, query builder, dobra dokumentacija. Ekipa ima izkušnje z raw SQL in Sequelize.

Odločitev

Izbrali smo Prisma ORM. 92 modelov v schema.prisma, migracije prek prisma migrate, type-safe queries prek Prisma Client.

Alternative

Alternativa	Pros	Cons	Zakaj zavrnjena
Drizzle ORM	Najboljša performanca, minimal	Manj funkcij, manj dokumentacije	Premalo zrelo (2025)
TypeORM	Popular, decorated entities	Težek, bug-ovit, slaba DX	Slabe izkušnje v preteklosti
Sequelize	Zrel, popularen	TypeScript šibek, počasen	Slaba TypeScript podpora
Raw SQL (pg)	Polna kontrola, hitro	Brez type safety, ročne migracije	Preveč ročno za 92 modelov
Kysely	Type-safe query builder	Brez migracij, brez generatorja	Manj funkcij kot Prisma

Tabela 4.1: Primerjava alternativ za ADR-003

Posledice

- **Good:** Type-safe queries (TypeScript inferira tipe iz sheme), avtomatske migracije, odlična DX
- **Good:** Prisma Studio za debug, Prisma Client je lightweight, dobra dokumentacija
- **Good:** 92 modelov z relacijami brez ročnega SQL - ogromna prihranek časa
- **Bad:** Prisma Client dodaja ~3MB bundle size (server-side, a vseeno)
- **Bad:** N+1 query problem (mitigiramo z include/select)
- **Bad:** Prisma migrate ima bug-e z neon tehnologijo (workaround: prisma db push)

5. ADR-004: Multi-tenant arhitektura

ADR-004	Multi-tenant arhitektura z locationId izolacijo	Accepted
Datum: 2025-02-01		

Kontekst

RestaurantOS mora podpirati verige restavracij z več lokacijami (npr. 5 lokacij = 5 tenantov). Vsaka lokacija mora imeti ločene podatke (naročila, zaloge, osebje), a skupne konfiguracije (meni, cene). Super-admin (PIN 5555) mora videti vse lokacije.

Zahteve: stroga izolacija podatkov, skupna koda, enostavno dodajanje novih lokacij, audit log za cross-branch operacije.

Odločitev

Izbrali smo "shared database, shared schema" multi-tenant arhitekturo. Vseh 8 ključnih tabel (orders, payments, inventory, employees, itd.) ima locationId stolpec. Aplikacijska plast (requireAuth + middleware) vsako query samodejno filtrira z locationId.

```
// Prisma query z avtomatsko filtracijo:
const orders = await db.order.findMany({
  where: {
    locationId: authResult.session.locationId, // iz session-a
    // ... drugi pogoji
  },
});

// Super-admin (PIN 5555) lahko izpusti locationId:
if (authResult.session.role === 'super_admin') {
  // query brez locationId filtra
}
```

Alternative

Alternativa	Pros	Cons	Zakaj zavrnjena
Database-per-tenant	Najboljša izolacija	Drago, kompleksno za migracije	Predrago za naš model
Schema-per-tenant	Dobra izolacija, ena baza	Kompleksne migracije, connection pooling	Preveč kompleksno
Row-level security (RLS)	DB-level izolacija	Težko debug, Neon podpira šibko	Premalo zrelo na Neon
Single-tenant (ločena instanca)	Enostavno	Drago, težko upravljati	Ne ustreza SaaS modelu

Posledice

- **Good:** Enostavno dodajanje novih lokacij (samo nov Location zapis), nizki stroški (ena baza)
- **Good:** Skupna koda in shema, enostavne migracije
- **Good:** Super-admin lahko vidi vse lokacije (cross-branch poročila, audit log)
- **Bad:** Aplikacijska plast mora VEDNO dodati locationId filter (rizenje za bug-e)
- **Bad:** Eventualna izolacijska napaka če developer pozabi filter (mitigiramo z requireAuth middleware)
- **Bad:** Težje za analytics (agregacija čez tenant-e zahteva skrbne query-je)

6. ADR-005: PIN-based auth

ADR-005	PIN-based auth namesto JWT/session	Accepted
Datum: 2025-02-10		

Kontekst

Restavracijsko osebje (natakarji, kuharji) se mora hitro prijaviti v POS. Uporaba email+geslo je prepočasna (45+ sekund na začetku izmene). PIN (4 številke) je hitro vnesti na tablici. Vendar PIN šibek za varnost.

Zahteve: hitra prijava (<5 sekund), varnost (PIN ne sme biti raw v bazi), podpora za offline (PIN veljaven tudi brez interneta), enostavno upravljanje (admin lahko resetira PIN).

Odločitev

Izbrali smo PIN-based auth z bcrypt + HMAC-SHA256 hashiranjem. PIN (4 številke) se hashira z bcrypt (10 rounds) in dodatno zaščiti z HMAC-SHA256. Session token (JWT-like) se generira po prijavi in velja 8 ur.

```
// PIN hashiranje (pri ustvarjanju uporabnika):
const saltRounds = 10;
const hmacKey = process.env.PIN_HMAC_KEY;
const hmacPin = crypto.createHmac('sha256', hmacKey).update(pin).digest('hex');
const hashedPin = await bcrypt.hash(hmacPin, saltRounds);

// PIN verifikacija (pri prijavi):
const hmacPin = crypto.createHmac('sha256', hmacKey).update(inputPin).digest('hex');
const valid = await bcrypt.compare(hmacPin, storedHashedPin);

// Session generacija (po uspešni prijavi):
const token = jwt.sign(
  { employeeId, role, locationId },
  process.env.NEXTAUTH_SECRET,
  { expiresIn: '8h' }
);
```

Alternative

Alternativa	Pros	Cons	Zakaj zavrnjena
Email + geslo	Standard, varno	Počasno (45s), keyboard potrebno	Prepočasno za POS
Biometric (FaceID)	Hitro, varno	Hardware zahteva, draga implementacija	Premalo tablic s FaceID
RFID kartica	Hitro (tap)	Hardware, kartice se izgubijo	Dodaten strošek hardware
Magic link (email)	Brez gesla	Potreben email, počasno	Ni ustreza za POS

Alternativa	Pros	Cons	Zakaj zavrnjena
SSO (Google/Microsoft)	Varno, enostavno	Vsak uporabnik rabi Google račun	Neprimerno za natakraje

Tabela 6.1: Primerjava auth alternativ

Posledice

- **Good:** Hitra prijava (<5s), enostavno za osebje, deluje offline (PIN se preverja lokalno)
- **Good:** PIN je hashiran (bcrypt + HMAC), ne moremo ga dehashirati
- **Good:** Admin lahko resetira PIN kadarkoli
- **Bad:** PIN je šibek (10.000 kombinacij) - mitigiramo z rate limiting (5 poskusov/15 min)
- **Bad:** Če PIN-HMAC-Key pušča, so vsi PIN-i kompromitirani (rotate ob incidentu)
- **Bad:** Session poteče po 8h (delovna izmena je lahko daljša - mitigiramo z refresh token)

7. ADR-006: FURS OpenSSL CLI + Node crypto

ADR-006	FURS modul z OpenSSL CLI + Node crypto fallback	Accepted
Datum: 2025-02-15		

Kontekst

FURS (slovenski davčni sistem) zahteva PKCS12 certifikate za davčno potrjevanje računov. Node.js crypto modul v standalone načinu ne podpira polnega PKCS12 parsing-a (posebej za FURS specifične certifikate). Potreben je zanesljiv način za ekstrakcijo privatnega ključa iz .p12 datoteke.

Zahteve: zanesljiva ekstrakcija privatnega ključa, podpora za FURS certifikate, varno geslo handling, delovanje na Vercel (serverless).

Odločitev

Izbrali smo hibridni pristop: OpenSSL CLI kot primarna metoda + Node.js crypto kot fallback. OpenSSL CLI je najbolj zanesljiv za FURS certifikate, Node crypto pa deluje kjer CLI ni na voljo.

```
// Primarna metoda: OpenSSL CLI
const pemKey = execFileSync('openssl', [
  'pkcs12', '-in', certPath, '-nocerts', '-nodes',
  '-passin', `pass:${password}`,
], { encoding: 'utf8', timeout: 10000 }).trim();

// Fallback: Node.js crypto (če OpenSSL ni na voljo)
if (!pemKey || !pemKey.includes('BEGIN')) {
  return tryNodeCryptoPKCS12(certPath, password);
}
```

Alternative

Alternativa	Pros	Cons	Zakaj zavrnjena
Samo Node.js crypto	Brez zunanjih deps	Ne deluje za vse FURS cert-e	Nezanesljivo
Samo OpenSSL CLI	Najbolj zanesljivo	Zahteva openssl v PATH (Vercel?)	Vercel ima openssl, a fallback vseeno
forge (pure JS)	Brez zunanjih deps	Počasen, bug-i z nekaterimi cert-i	Nezanesljivo za FURS

Alternativa	Pros	Cons	Zakaj zavrnjena
Cloud function (AWS Lambda)	Polna kontrola	Dodaten cloud, latenco	Predrago in kompleksno

Tabela 7.1: Primerjava PKCS12 parsing strategij

Posledice

- **Good:** Zanesljiva ekstrakcija ključa za vse FURS certifikate (OpenSSL je standard)
- **Good:** Fallback omogoča delovanje tudi brez OpenSSL (Node crypto)
- **Good:** execFileSync preprečuje shell injection (argumenti so array, ne string)
- **Bad:** Odvisnost od openssl v PATH (Vercel ima, a self-host bi bil problem)
- **Bad:** 10s timeout (lahko zablokira request) - mitigiramo z cacheanjem ključa
- **Bad:** Dve kodi path-a za vzdrževanje (CLI + Node crypto)

8. ADR-007: Service Worker offline

ADR-007	Service Worker za offline mode (IndexedDB queue)	Accepted
Datum: 2025-03-01		

Kontekst

Restavracije imajo pogosto nestabilen internet. Natakarji morajo lahko sprejemati naročila tudi brez interneta. Ko povezava povrne, se naročila sinhronizirajo. FURS dovoli 48h zamik pri potrjevanju.

Zahteve: offline naročila, offline plačila (gotovina), offline FURS queue, avtomatska sinhronizacija, delovanje na iOS in Android.

Odločitev

Izbrali smo Service Worker (PWA) z IndexedDB queue in Background Sync API. Service Worker (sw.js v9) cach-a statiko (cache-first) in API (network-first z fallback). Naročila se shranjujejo v IndexedDB in sinhronizirajo prek Background Sync.

```
// Service Worker cache strategije:
// 1. Static assets (/ , manifest.json, icons) → cache-first
// 2. API (/api/menus, /api/tables) → network-first, fallback na cache
// 3. Občutljivi API (/api/orders/*/pay, /api/furs) → NIKOLI cache

// Offline orders queue (IndexedDB):
async function queueOrder(order) {
  const db = await openDB('restaurantos', 1);
  await db.add('orders', { ...order, status: 'pending', createdAt: Date.now() });
}

// Background Sync (ko povezava povrne):
self.addEventListener('sync', (event) => {
  if (event.tag === 'sync-orders') {
    event.waitUntil(syncOrders());
  }
});
```

Alternative

Alternativa	Pros	Cons	Zakaj zavrnjena
Native app (iOS/Android)	Najboljša offline izkušnja	2 kodi bazi, drag razvoj	Predrago in počasno
React Native	Ena koda, native perf	Še vedno 2 build-a, kompleksno	Prekompleksno za našo ekipo

Alternativa	Pros	Cons	Zakaj zavrnjena
Local SQLite (Capacitor)	Močna offline baza	Capacitor setup, še vedno "native"	Ne uporabljamo Capacitor
Samo cache (brez queue)	Enostavno	Ne deluje za write operacije	Ne zadostuje za naročila
No offline (online-only)	Enostavno	Ne deluje brez interneta	Ne sprejemljivo za restavracije

Tabela 8.1: Primerjava offline strategij

Posledice

- **Good:** Deluje na vseh platformah (iOS Safari 16.4+, Android Chrome, desktop)
- **Good:** Ena koda (spletna), brez native app store procesa
- **Good:** IndexedDB je močna (lahko shranimo tisoče naročil)
- **Good:** Background Sync avtomatsko sinhronizira (tudi ko je app zaprta)
- **Bad:** iOS < 16.4 ne podpira push notifications (mitigiramo z SMS fallback)
- **Bad:** Service Worker cache lahko povzroči "stale data" (mitigiramo z versioning v10)
- **Bad:** Debugging Service Worker je težko (DevTools je edino orodje)

9. ADR-008: Cascade palette V2

ADR-008	Cascade palette V2 za design system	Accepted
Datum: 2025-03-15		

Kontekst

RestaurantOS potrebuje dosledno barvno paletu, ki: (1) ustreza restavracijski tematiki (topla, vabljiva), (2) je dosledna čez vse komponente, (3) je harmonična (ne random barve), (4) je dostopna (WCAG AA contrast).

Zahteve: 60-30-10 razmerje (background-surface-accent), nizka saturacija za velike površine, visoka saturacija samo za accents, matematično harmonična.

Odločitev

Izbrali smo Cascade palette V2 z 5 nivoji (XL, L, M, S, XS) glede na saturacijo in area. Generirana z design_engine.py palette.cascade.

Tier	Area	Saturacija	Barva	Uporaba
XL	>50%	$S \leq 0.08$	#f3f2f1 (kremna)	Page background
L	20-50%	$S \leq 0.15$	#eeedea (svetla)	Cards, surfaces
M	5-20%	$S \leq 0.30$	#685f46 (zemeljska)	Headers, structural fills
S	1-5%	$S \leq 0.50$	#d1c9b3 (peščena)	Borders, icons
XS	<1%	$S \leq 0.75$	#86702b (zlata)	Accents, badges, emphasis

Tabela 9.1: Cascade palette V2 nivoji

Alternative

Alternativa	Pros	Cons	Zakaj zavrnjena
Tailwind default palette	Enostavno, znano	Niso harmonične, random saturacija	Ne dosledno
Material Design palette	Zrelo, dobra dokumentacija	Preveč 'tech', ne restavracijska	Ne ustreza tematiki
Custom hand-picked	Polna kontrola	Težko harmonično, subjektivno	Ne matematično
Monochrome (siva)	Minimalistično	Dolgočasno, brez "osebnosti"	Ne primerno za restavracije

Alternativa	Pros	Cons	Zakaj zavrnjena
Brand-led (logo barve)	Skladno z brand	Limitira izbire, ne harmonično	Premalo fleksibilno

Tabela 9.2: Primerjava paletnih strategij

Posledice

- **Good:** Matematično harmonična (HSL-bazirana), dosledna čez vse komponente
- **Good:** Topla zemeljska paleta ustreza restavracijski tematiki (razlikuje od tekmovalcev)
- **Good:** 60-30-10 razmerje preprečuje prekomerno uporabo accent barv
- **Good:** Generirana avtomatsko (palette.cascade) - reproducibilna
- **Bad:** Topla paleta je manj "korporativna" (lahko je izziv za enterprise segment)
- **Bad:** Design system ni formalno dokumentiran (Storybook manjka - P1 prioriteta)

10. ADR-009: Stripe plačilni gateway

ADR-009	Stripe kot primarni plačilni gateway	Accepted
Datum: 2025-04-01		

Kontekst

RestaurantOS potrebuje integriran plačilni gateway za kartična plačila v POS. Zahteve: PCI compliance (ne shranjujemo kartičnih podatkov), podpora za EU kartice (3-D Secure/SCA), webhook za asinhrona potrdila, refund support, dobra dokumentacija.

Omejitve: slovenski trg (EUR valuta), ekipa pozna Stripe, budget za transakcijske provizije.

Odločitev

Izbrali smo Stripe kot primarni plačilni gateway. Uporabljamo Stripe Elements (PCI-compliant), PaymentIntent API, webhook za event processing. Implementirana je StripeGateway class (286 vrstic) z authorize/capture/refund.

Alternative

Alternativa	Pros	Cons	Zakaj zavrnjena
SumUp	EU friendly, fizični terminali	Šibkejši API, manj funkcij	Manj funkcij kot Stripe
Braintree (PayPal)	PayPal + kartice	PayPal lock-in, šibkejši EU	Slabša EU podpora
Adyen	Enterprise, multi-currency	Drago, kompleksno setup	Predrago za začetek
PayPal direct	Znana blagovna znamka	Slab UX, PayPal-only stranke	Ne dovolj fleksibilno
Local SI processor (NLB)	Lokalno, brez provizije	Stara API-ja, brez webhooks	Neprimerno za sodoben POS

Tabela 10.1: Primerjava plačilnih gatewayov

Posledice

- **Good:** PCI-compliant (kartice nikoli ne pridejo na naš server - Stripe Elements)
- **Good:** Odlična dokumentacija, dobra EU podpora, 3-D Secure (SCA) built-in
- **Good:** Webhook za asinhrona potrdila (payment_intent.succeeded, itd.)
- **Good:** Enostaven refund (delni in polni), dober dashboard

- **Bad:** Transakcijska provizija 1.5% + 0.18 EUR (višja od lokalnih)
- **Bad:** Vendor lock-in (Stripe API je specifičen)
- **Bad:** Disputes se obračunavajo po Stripe pravilih (lahko 7 dni čaka)

11. ADR-010: Chain hash audit log

ADR-010	Chain hash audit log (SHA-256, nepopravljiv)	Accepted
Datum: 2025-04-15		

Kontekst

RestaurantOS mora imeti nepopravljiv audit log za: FURS operacije (zakonska zahteva), plačilne operacije (PCI audit), admin operacije (security). Auditorji morajo lahko preverili, da log ni bil popravljen.

Zahteve: nepopravljiv (chain hash), časovno sledljiv, varčen pred DB izgubo, enostaven za verificirati.

Odločitev

Izbrali smo chain hash audit log. Vsaka log vrstica vsebuje: prejšnji hash, vsebino, trenutni hash (SHA-256 prejšnji + vsebina). Prva vrstica (genesis) ima fixed hash.

Verifikacija: rechain od genesis do zadnje vrstice.

```
// AuditLog model (Prisma):
model AuditLog {
  id          String   @id @default(cuid())
  timestamp   DateTime @default(now())
  employeeId  String
  action      String   // "furs.verify", "payment.create", itd.
  entityId    String
  ipAddress   String?
  metadata    Json?
  previousHash String
  currentHash String   // SHA-256(previousHash + content)

  @@index([timestamp])
  @@index([employeeId])
}

// Generacija hash:
const content = JSON.stringify({ timestamp, employeeId, action, entityId, metadata });
const currentHash = crypto.createHash('sha256')
  .update(previousHash + content)
  .digest('hex');

// Verifikacija (rechain):
let prevHash = GENESIS_HASH;
const logs = await db.auditLog.findMany({ orderBy: { timestamp: 'asc' } });
for (const log of logs) {
  const expectedHash = crypto.createHash('sha256')
    .update(prevHash + log.content)
    .digest('hex');
  if (log.currentHash !== expectedHash) {
    throw new Error(`Audit log compromised at ${log.id}`);
  }
  prevHash = log.currentHash;
}
```

}

Alternative

Alternativa	Pros	Cons	Zakaj zavrnjena
Simple DB tabela	Enostavno	Popravljivo (admin lahko uredi)	Ne nepopravljivo
Append-only DB	DB-level zaščita	Težko implementirati na Neon	Ne podprto na Neon
Blockchain (Ethereum)	Decentralizirano	Predrago, počasno, overkill	Excessive
External service (Splunk)	Profesionalno	Drago, dodaten cloud	Predrago za našo skalo
File-based log (JSON)	Enostavno	Popravljivo, težko query	Ne queryable

Tabela 11.1: Primerjava audit log strategij

Posledice

- **Good:** Nepopravljiv (vsak popravek prekine chain - detekcija)
- **Good:** Verifiable (kdorkoli lahko rechain in preveri integriteto)
- **Good:** Združljivo z GDPR (log je metadata, ne osebni podatki)
- **Good:** Enostavno za implementirati (crypto modul v Node.js)
- **Bad:** Verifikacija je $O(n)$ (počasno za milijone vrstic) - mitigiramo z batch verify
- **Bad:** Če ena vrstica izgine, chain prekinjen (mitigiramo z backup)
- **Bad:** Storage raste (vsaka vrstica ~500 bytes) - mitigiramo z arhiviranjem

12. ADR-011: Vercel hosting

ADR-011	Vercel kot hosting platforma	Accepted
Datum: 2025-05-01		

Kontekst

RestaurantOS potrebuje hosting za Next.js aplikacijo. Zahteve: avtomatski deploy iz Git, EU hosting (GDPR), serverless functions, edge caching, enostaven rollback, dobra observability.

Omejitve: budget <50 EUR/mes v prvem letu, ekipa brez DevOps izkušenj, prioriteta enostavnost.

Odločitev

Izbrali smo Vercel kot hosting platformo. Pro plan (20 USD/mes) za produkcijo, avtomatski deploy iz GitHub main branch, EU region (Frankfurt), serverless functions za API.

Alternative

Alternativa	Pros	Cons	Zakaj zavrnjena
Netlify	Enostavno, dobra DX	Šibkejši za Next.js, manj funkcij	Vercel je Next.js creator
AWS Amplify	AWS ekosistem	Kompleksno, drago, slaba DX	Prekompleksno
Self-hosted (Docker + VPS)	Polna kontrola, ceneje	Vzdrževanje, scaling naša odgovornost	Ni dovolj časa za ops
Railway	Enostavno, ceneje	Manj funkcij, šibkejši za Next.js	Manj zrelo
Cloudflare Pages	Hitro, ceneje	Šibkejši za Next.js App Router	Kompatibilnost vprašljiva

Tabela 12.1: Primerjava hosting platform

Posledice

- **Good:** Avtomatski deploy iz Git (push na main = deploy), preview deployments za PR
- **Good:** EU region (Frankfurt) za GDPR skladnost
- **Good:** Enostaven rollback (vsak deployment ima URL, promote to production)
- **Good:** Serverless functions (auto-scale, pay-per-use)

- **Good:** Built-in analytics, logs, observability
- **Bad:** Vendor lock-in (Next.js specifične funkcije)
- **Bad:** Function execution limit (10s na free, 60s na Pro) - problem za dolge operacije
- **Bad:** Bandwidth limit (100 GB/mes na Pro) - bomo morali upgrade pri rasti

13. ADR-012: next-intl za i18n

ADR-012	next-intl za i18n (5 jezikov)	Accepted
Datum: 2025-05-15		

Kontekst

RestaurantOS mora podpirati 5 jezikov: slovenščina (primarni), angleščina, italijanščina, hrvaščina, nemščina. Zahteve: prevodi v JSON datotekah, lazy loading, formatiranje števil/ datumov/ valut, RTL podpora (za arabščino v prihodnosti).

Omejitve: integracija z Next.js App Router, dober TypeScript support, enostaven upload novih prevodov.

Odločitev

Izbrali smo **next-intl** za internacionalizacijo. Prevodi v `src/messages/{sl,en,it,hr,de}.json`. Locale detection iz URL-ja (`/sl`, `/en`, itd.) ali cookie.

Alternative

Alternativa	Pros	Cons	Zakaj zavrnjena
react-intl (FormatJS)	Zrel, popularen	Šibek Next.js App Router support	Slaba App Router integracija
i18next	Najbolj popularen	Kompleksen setup, šibek TS	Prekompleksno za naše potrebe
Lingui	Compile-time, hitro	Manj popularen, manj docs	Premajhna skupnost
Custom solution	Polna kontrola	Veliko dela, bug-i	Preveč ročno
react-i18next	Zrel, popularen	Kompleksen, šibek App Router	Enako kot i18next

Tabela 13.1: Primerjava i18n knjižnic

Posledice

- **Good:** Enostavna integracija z Next.js App Router (server in client components)
- **Good:** TypeScript support (tipi za prevode)
- **Good:** Lazy loading prevodov (samo aktivni jezik se naloži)
- **Good:** Formatiranje števil, datumov, valut (ICU MessageFormat)
- **Bad:** next-intl je relativno nov (manj skupnost kot react-intl)

- **Bad:** RTL podpora je šibka (če bomo dodali arabščino)
- **Bad:** Translation management ni vgrajen (uporabljamo Lokalise ali Crowdin)

14. Skupne posledice in naslednji koraki

Ta sekca povzema skupne posledice vseh 12 ADR-jev in identificira področja, ki jih je treba ponovno pretehtati.

14.1 Skupne prednosti

- **Modern stack:** Next.js 16 + React 19 + TypeScript + Prisma + Neon = sodobna, hitra, scalable arhitektura
- **EU/GDPR skladnost:** Vercel EU + Neon EU + chain hash audit log = GDPR ready
- **Multi-tenant SaaS:** Ena koda, več strank, nizki stroški (Neon serverless)
- **Offline-first:** Service Worker + IndexedDB = deluje brez interneta
- **Security:** PIN bcrypt + HMAC, PCI-compliant Stripe, audit log
- **DX:** TypeScript + Prisma + Next.js = odlična developer izkušnja

14.2 Skupne slabosti

- **Vendor lock-in:** Vercel + Neon + Stripe = težko selimo na alternativne
- **Šibka mobilna izkušnja:** PWA (ne native) - omejitve na iOS < 16.4
- **Multi-tenant risk:** Aplikacijska plast mora VEDNO dodati locationId filter
- **Function execution limit:** Vercel 60s timeout za dolge operacije
- **Brez formalnega design sistema:** Storybook manjka (P1 prioriteta)

14.3 ADR-ji za ponovno pretehtavanje

ADR	Zakaj ponovno pretehtati	Kdaj	Mogoča sprememba
ADR-005 (PIN auth)	Šibka varnost (10k kombinacij)	Ko se pojavi fraud incident	Dodaj 2FA za admin
ADR-007 (Service Worker)	iOS < 16.4 omejitve	Q1 2026 (iOS update)	Native app za iOS
ADR-009 (Stripe)	Visoke provizije za velike stranke	Ko >1M EUR letni volumen	Negotiat e z Adyen
ADR-011 (Vercel)	Bandwidth/function limit	Ko presežemo 100GB/mes	Self-host ed z Docker
ADR-012 (next-intl)	RTL podpora šibka	Če dodamo arabščino	Migratio n na react-intl

Tabela 14.1: ADR-ji za ponovno pretehtavanje

14.4 Postopek za nove ADR-je

Ko se pojavi nova ključna tehnična odločitev, se ustvari nov ADR:

- 1. Identificiraj problem (zakaj odločamo)
- 2. Zapiši kontekst (zahteve, omejitve)
- 3. Razmisli o alternativah (vsaj 3)
- 4. Izberi in utemelji (zakaj ta, ne druga)
- 5. Zapiši posledice (good/bad/neutral)
- 6. Daj na review (Tech Lead + 1 developer)
- 7. Po odobritvi - dodaj v to zbirko (ADR-XXX)
- 8. Če se odločitev spremeni - označi star ADR kot "Superseded by ADR-YYY"

ADR KULTURA

ADR-ji niso birokracija - so neprecenljivo znanje. Nov član ekipe lahko v 1 uri razume "zakaj" za vsako ključno odločitev. Brez ADR-jev se znanje izgubi ko člani odidejo. Piši ADR za vsako odločitev, ki bi jo lahko preklical v prihodnosti.